PulseOS

1. Introduction

Pulse OS is a personal command-centre dashboard that brings the many separate services people check every day into a single, calm, unscrolling interface. Where a typical routine means opening one app for the weather, another for the calendar, another for news, markets, sports, music and finances, Pulse OS gathers these domains into one cohesive "quiet-glass" surface. The aim of the project is to reduce the friction and noise of jumping between applications by presenting everything the user needs in one glance, organised into a clean bento-style layout that never requires the page to scroll.

The system is built as a monorepo with a React and Vite frontend and a Node.js and Express backend that acts as a secure gateway to a wide range of third-party providers. The backend follows a clean, layered architecture that separates routing, controllers, services and provider integrations, which keeps each external service isolated and the codebase maintainable. The frontend mirrors this separation with feature components, state hooks and a single typed API client. Additional features include a provider-swappable AI assistant with read and write access to the user's own data, live news streaming, real-time market and sports data, calendar synchronisation across Google and Apple, on-device music playback control, and request rate limiting with hard usage caps to keep the application inside free-tier limits.

1.5 Project Function and What It Does

Pulse OS is an all-in-one personal dashboard designed to help a user manage their whole day from one screen. From the user's point of view, the app opens to a personalised home view that greets them by name and immediately shows the weather, a multi-day forecast, their upcoming calendar events, their next work shift, and a launchpad of their most-used applications. Instead of checking a weather app, a calendar app, a news app and a banking app separately, Pulse OS presents these together in one simple experience. Beyond the home view, the app supports the user across distinct areas: a Life Hub for schedules, to-dos, projects and daily habits; a Markets & News area with live television news, a stock ticker, and configurable news, sports and market feeds; a Music player; a Travel planner; and a Finance overview. A built-in AI assistant sits at the centre of the navigation and can read the user's live data and make changes on their behalf, such as adding tasks or answering questions about their schedule. Overall, the main function of the project is to make daily life easier to manage by combining information, planning and personal services into a single, unified platform.

2. Tech Stack Used

- **JavaScript (ES Modules)**
- **React**
- **Vite**
- **Tailwind CSS**
- **Node.js**
- **Express.js**
- **Firebase (Cloud Functions and Hosting ready)**
- **OpenWeather API**
- **Finnhub API**
- **CoinGecko API**
- **GNews API**
- **Football-Data.org, balldontlie and Jolpica (Ergast) APIs**
- **Google Calendar API**
- **Apple iCloud Calendar (CalDAV)**
- **Spotify Web API and Web Playback SDK**
- **Google Gemini AI (with OpenAI and Anthropic Claude as swappable providers)**
- **hls.js**
- **lucide-react**

React and Vite were used to build the frontend because they allow a fast, modern, component-driven interface with a very quick development feedback loop, which suited a project made up of many independent feature cards that each manage their own data. Tailwind CSS was used for styling because it made it possible to build the consistent "quiet-glass" visual language — frosted cards, soft glows and a single-viewport bento layout — quickly and without a large custom stylesheet. lucide-react provided a clean, consistent icon set for the navigation dock and cards, while hls.js was used to stream live television news directly inside the browser. On the backend, Node.js and Express were used to build a clear API gateway, with each domain given its own route, controller and service so that request handling stays separate from the logic that talks to each external provider.

Firebase was chosen as the deployment target because the backend is structured so the Express application can be wrapped by Cloud Functions without changing any route or service, and Firebase Hosting can serve the built frontend. A deliberately wide set of third-party APIs was integrated to cover each domain of the dashboard: OpenWeather for weather and forecasts, Finnhub and CoinGecko for stocks and cryptocurrency, GNews together with RSS feeds for headline and local news, and a combination of Football-Data.org, balldontlie and the key-free Jolpica (Ergast) service for football, NBA, NFL and Formula 1 data. Google Calendar and Apple iCloud (over CalDAV) were integrated so the user's real events appear in the app, and the Spotify Web API and Web Playback SDK were used to control music playback on the device. For

the AI assistant, Google Gemini was chosen as the default provider because it offers the most generous free tier and supports function calling, with OpenAI and Anthropic Claude available as drop-in alternatives selectable per request.

.

3. Architecture Used

Pulse OS follows a feature-based, layered architecture across a single monorepo managed with npm workspaces, with the frontend and backend kept as separate packages. It does not rely on a heavy framework, but responsibilities are clearly separated. On the backend, every incoming request passes through a defined chain: a route file maps the URL to a controller, the controller validates input and delegates to a service, and the service coordinates one or more providers that each wrap a single external API. This means, for example, that the weather route never talks to OpenWeather directly — it goes through a weather controller, a weather service, and finally an OpenWeather provider — so any provider can be swapped or extended without touching the rest of the code.

Supporting this are a set of cross-cutting layers. A single configuration module reads every environment variable and exposes a typed, central view of the app's settings, so nothing else in the codebase reads process variables directly. A middleware layer handles CORS, request rate limiting and centralised error handling, turning typed application errors into clean JSON responses. A utilities layer provides shared tools including a time-to-live cache that serves stale data rather than failing when a provider is rate-limited, a persisted token store for OAuth sessions, and a persisted usage meter that enforces hard spend caps on the metered AI providers. Importantly, the Express application is constructed separately from the call that starts the server, which is what makes the backend ready to run either locally or inside Firebase Cloud Functions with no code changes.

On the frontend, the interface is organised into feature components, each representing a card or screen, backed by dedicated state hooks such as those for weather, calendar events, life data, market data, finances, travel and the Spotify player. All communication with the backend flows through one typed API client, which is the single place the frontend talks to real integrations. Some feature areas, specifically Travel and Finance, are intentionally kept entirely local to the device and do not call the backend at all. This layered separation on both sides of the application makes each part easy to understand, test and maintain, because every feature has a clear and isolated purpose.

4. Full Functionality with Technical Implementation

The application is organised around a persistent navigation dock that gives access to seven areas: Home, Life Hub, Markets & News, the AI assistant, Music, Travel and Finance. The dock acts as the central shell of the interface, switching between feature views while keeping the calm, single-

viewport layout consistent throughout. A settings panel, reachable from the home view, collects the personal details that the rest of the app depends on, including the user's display name, their location and their followed sports teams. The display name is reused in the greeting and the AI assistant, the location drives local weather and local news, and the selected teams populate the sports card. This gives the project a controlled, personalised entry point where later features draw on a shared set of preferences rather than asking for the same information repeatedly.

The Home screen provides a summary of the user's day. It greets the user by name and shows the current weather and a multi-day forecast for their location, a launchpad of their chosen applications, a timeline of upcoming calendar events, and their next work shift. Weather is retrieved through the weather hook, which calls the backend weather service and in turn the OpenWeather provider, returning the current conditions, a short human-readable summary and the forecast. The upcoming events are drawn from the user's connected calendars, and the layout is designed so that all of this fits within a single screen without scrolling, acting as a launch point into the rest of the app.

The Launchpad is a distinctive feature that lets the user open real applications on their machine directly from the dashboard. The backend inspects the system's application directories to list installed apps, extracts each application's own icon and serves it as an image, and opens an application on request. Because this feature works directly with the local operating system, it is designed to run when the backend is hosted on the user's own machine. Through a picker interface, the user can choose up to ten applications to pin to the launchpad, search the full list of installed apps, and then open any of them with a single tap; where an app cannot be opened directly, the system falls back to a related web link.

Calendar functionality is handled through a dedicated calendar service that supports three sources: Google Calendar over OAuth, Apple iCloud through the CalDAV protocol, and subscribed public .ics feeds. A connection panel lets the user link or disconnect each provider, paste a public calendar feed, and toggle the visibility of individual calendars. Once connected, events sync into the app and feed both the home screen's upcoming timeline and the Life Hub schedule. Events are watched so that the interface reflects the user's real, current agenda, and the layered provider design means each calendar source is handled by its own provider behind a common service interface.

The Life Hub is the planning area of the application. It presents the day's schedule alongside a to-do list, a set of projects with their own nested tasks and completion progress, and a list of daily habits with a running completion count. The life data hook manages this state and its persistence, allowing tasks and habits to be created, checked off and cleared, with projects tracking how many of their tasks are complete. This gives the user a lightweight but structured place to manage their commitments, with the schedule portion drawing on the same connected calendar data used elsewhere in the app.

The Markets & News area combines several real-time data sources into one screen. A live television news stream is played directly in the browser using hls.js, with channel controls and a link to open the source site. Above it, a continuously scrolling ticker shows live prices for a watchlist

of stocks and cryptocurrencies, retrieved through the market data hook from the backend's Finnhub and CoinGecko providers. A configurable feed panel lets the user switch between top headlines, local news based on their saved location, sports and market views. News is gathered on the backend through the GNews provider supplemented by RSS feeds, with local news resolved from the user's town or county. A compact weather strip repeats the current conditions so the user keeps that context while reading.

The Sports functionality supports football, the NBA, the NFL and Formula 1. Depending on the selected team or competition, the app shows the next fixture and a full standings table, such as a Premier League table with the user's team highlighted, or the Formula 1 drivers' and constructors' championships with team badges and points. On the backend these are served by separate providers — Football-Data.org for football, balldontlie for the NBA and NFL, and the key-free Jolpica (Ergast) service for Formula 1 — each wrapped by its own service so the different data shapes are normalised before reaching the interface. The stocks view provides a dedicated watchlist showing each instrument's price and percentage change, with a summary of how many are up or down, sourced live from Finnhub.

The Music area lets the user control playback on their device through Spotify. Using the Spotify Web API and Web Playback SDK, the player hook manages authentication, transfers playback to the current device, and exposes a now-playing view with track details, a progress bar and transport controls. The user can browse their playlists and search the full Spotify catalogue, with results shown alongside the player so a track can be found and played without leaving the screen. As with the other integrations, all Spotify communication is proxied through the backend so that secret credentials never reach the client.

The Travel and Finance areas are intentionally kept local to the device and do not depend on the backend. The Travel planner counts down to an upcoming trip and organises it into cards for flight and hotel details, a live local-time clock and weather for the destination, a currency converter with quick-amount presets, a packing checklist with progress, and a day-by-day itinerary. The Finance overview presents the user's liquid balance, net worth, income, expenses and cashflow, a monthly spending trend drawn as a line chart, a list of accounts, upcoming bills and savings goals with progress toward each target. Both areas manage their own state through dedicated stores and persist it on the device, keeping personal financial and travel information private to the user's own machine.

At the centre of the navigation sits the AI assistant, which is more than a simple chat feature. On the frontend it runs an agent loop: the conversation is sent to the model together with a set of tool definitions and a system prompt, and while the model responds with tool calls, the app executes each tool, feeds the result back, and continues until the model returns a final answer, up to a fixed number of steps. This allows the assistant to read the user's live data — such as their weather, calendar or tasks — and to make changes on their behalf, rather than only producing text. On the backend, the AI service resolves which provider to use, defaulting to Google Gemini but able to switch to OpenAI or Anthropic Claude per request, and each provider is wrapped so their different message and function-calling formats are handled behind a common interface.

Finally, the project includes a request rate limiting and usage capping system designed to keep the application safe to run and free to operate. Every API route passes through an in-memory rate limiter that caps requests per client to absorb runaway polling, with tighter limits on the AI route and on any request that changes state. Separately, a persisted usage meter enforces hard totals on the metered AI providers, checking a provider's daily and monthly request and token budgets before each call so a cap can be reached but never exceeded. The default Gemini budgets are deliberately set beneath the provider's free-tier limits, meaning the AI assistant can run without ever incurring a charge, and because the counters are written to disk they survive restarts rather than resetting. This shows that the project treats cost and reliability as first-class concerns rather than afterthoughts.